

School of Computer Science and Engineering

(Computer Science & Engineering - CSE)

Faculty of Engineering & Technology

Jain Global Campus, Kanakapura Taluk - 562112 Ramanagara District, Karnataka, India

2025-2026

(VII Semester)

A Project Report on

"Bank Fraud Detection"

PRODUCTION-READY MACHINE LEARNING-BASED BANK FRAUD DETECTION SYSTEM

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE & ENGINEERING - CSE

Submitted by

Raunak Kumar	23BTRCT109
Trina Mukherjee	23BTRCT137
Vibha Sabareesan	23BTRCT144
Tanisha Prajesh Doshi	23BTRCT135
Rakshith Gowda M	23BTRCT108

Under the guidance of

Dr. Ramya Dorai D

Associate Professor,

Department of Computer Science & Engineering – CSE,

School of Computer Science & Engineering Faculty of Engineering & Technology,

JAIN (Deemed to-be University).

Department of Computer Science & Engineering - CSE

School of Computer Science & Engineering Faculty of Engineering & Technology

Jain Global Campus, Kanakapura

Taluk - 562112 Ramanagara District, Karnataka, India

Department of Computer Science & Engineering - CSE

School of Computer Science & Engineering Faculty of

Engineering & Technology

Jain Global Campus, Kanakapura Taluk - 562112 Ramanagara District, Karnataka, India

CERTIFICATE

This is to certify that the project work titled "Bank Fraud Detection" (Production-Ready Machine Learning-Based Bank Fraud Detection System) is carried out by RAUNAK KUMAR (23btrct109), TRINA MUKHERJEE (23btrct137), TANISHA PRAJESH DOSHI (23btrct135), VIBHA SABAREESAN (23btrct144) AND RAKSHITH GOWDA M (23btrct108), Bonafide students of Bachelor of Technology at the School of Computer Science and Engineering - CSE, Faculty of Engineering & Technology, JAIN (Deemed-to-be University), Bangalore in partial fulfillment for the award of degree in Bachelor Technology in Computer Science & Engineering - CSE, during the year 2025-2026.

Dr. Ramya Dorai D

Project Guide
Associate Professor
Dept. of Computer Science &
Engineering - CSE
JAIN (Deemed to-be
University)

Date:

Dr. T.R. Mahesh

Program Head
Computer Engineering – CSE
School of Computer Science
& Engineering
JAIN (Deemed to-be
University)

Date:

Dr. Kamlesh Tiwari

Head of Department
Computer Science &
Engineering
Faculty of Engineering &
Technology
JAIN (Deemed-to-be
University)

Date:

Name of the Examiner

Examiner 1.

Signature of

Examiner 2.

DECLARATION

We, RAUNAK KUMAR, TRINA MUKHERJEE ,VIBHA SABAREESAN, TANISHA PRAJESH DOSHI, and RAKSHITH GOWDA M , students of 7th semester B.Tech in Computer Science & Engineering - CSE, at School of Engineering & Technology, Faculty of Engineering & Technology, JAIN (Deemed to-be University), hereby declare that the Project work titled "Bank Fraud Detection - Production-Ready Machine Learning-Based Bank Fraud Detection System “ has been carried out by us and submitted in partial fulfilment for the award of degree in Bachelor of Technology in Computer Science & Engineering - CSE during the academic year 2025-2026.

Further, the matter presented in the work has not been submitted previously by anybody for the award of any degree or any diploma to any other University, to the best of our knowledge and faith.

Raunak Kumar

(23BTRCT109)

Trina Mukherjee

(23BTRCT137)

Vibha Sabareesan

(23BTRCT144)

Tanisha Prajesh Doshi

(23BTRCT135)

Rakshith Gowda M

(23BTRCT108)

Place: Bangalore

Date:

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of a large number of individuals who have been responsible for the successful completion of this project work.

We take this opportunity to express our sincere gratitude to the Faculty of Engineering & Technology, JAIN (Deemed-to-be University) for providing us with an excellent opportunity to pursue our Bachelor's Degree.

*We are deeply grateful to **Dr. Chenraj Roy Chand, Chancellor**, for his tireless commitment to fostering excellence in teaching and research at Jain (Deemed-to-be University). We are also profoundly grateful to **Dr. Dinesh Nilkant, Pro Vice Chancellor**, for his unwavering support, and to **Dr. Jitendra Kumar Mishra, Registrar**, for his valuable guidance.*

*We extend our sincere gratitude to **Dr. Kamlesh Tiwari, Head of Department, School of Computer Science & Engineering, Faculty of Engineering & Technology**, for constant encouragement and expert advice. Additionally, I would like to express my appreciation to **Dr. Deepak Kumar Sinha, Deputy Director (Students & Industry Relations)**, for their invaluable contributions and support throughout this project. We also thank **Dr. T.R. Mahesh, Program Head, Computer Science & Engineering - CSE**, for providing the right academic guidance.*

*It is our immense pleasure to express heartfelt thanks to our project guide **Dr. Ramya Dorai D, Associate Professor, Dept. of Computer Science & Engineering - CSE**, for her valuable time, expert mentorship, and consistent support at every step of this work, which enabled the smooth progress and successful completion of this project. We would also like to thank all the staff members of Computer Science & Engineering - CSE for their continuous support, and our family and friends who provided us with all the requirements throughout the course.*

*I would like to thank our **Project Coordinator Dr. C.R.Raman**, and all the staff members of Computer Science & Engineering - CSE for their support.*

Signature of Students:

Raunak Kumar

(23BTRCT109)

Trina Mukherjee

(23BTRCT137)

Vibha Sabareesan

(23BTRCT144)

Tanisha Prajesh Doshi

(23BTRCT135)

Rakshith Gowda M

(23BTRCT108)

TABLE OF CONTENTS

CERTIFICATE	i-ii
DECLARATION	iii
ACKNOWLEDGEMENT	iv
TABLE OF CONTENTS	v-vi
LIST OF FIGURES	Vii
LIST OF TABLES	vii
ABSTRACT	viii
Chapter 1: Introduction	9
1.1 Background & Motivation	9
1.2 Objectives	10
1.3 Benefits of Research	10
Chapter 2: Literature Survey	11
2.1 Literature Review	11
2.1.1 Analysis of Literature	12
2.2 Problem Formulation and Proposed Work	12
2.2.1 Problem Statement	12-13
2.2.2 System Architecture / Model	13
2.2.3 Proposed Algorithms	13-14
2.2.4 Proposed Work	14
Chapter 3: System Analysis and Proposed Methodology	15
3.1 Introduction	15
3.2 System Architecture	15-16
3.3 Proposed Algorithms	16-17
3.4 Feature Engineering & Methodology	17-18

Chapter 4: System Design and Implementation	19
4.1 Introduction	19
4.2 System Design Principles	19
4.3 Dataset Design & Preprocessing	19
4.4 Model Architecture & Implementation	19
4.5 Flask Application Design	19-20
4.6 Implementation Workflow	20-21
Chapter 5: Tools and Technologies	22
5.1 Development Tools	22
5.2 Frontend Technologies	22
5.3 Backend Technologies	22
5.4 Security Technologies	22
Chapter 6: Software Requirement Specification (SRS)	23
6.1 Functional Requirements	23
6.2 Non-Functional Requirements	23
Chapter 7: Results and Discussion	24
7.1 Model Performance Metrics	24
7.2 Dashboard and Interface	24-26
7.3 Discussion of Findings	27
Chapter 8: Conclusions and Future Scope	28
8.1 Conclusions	28
8.2 Future Scope	28
References	29

LIST OF FIGURES

S.No	Figure Number	Page Number(s)
1	Fig-3.1 – System Architecture Diagram	16
2	Fig-7.2.1 – Real-Time Fraud Detection Dashboard	25
3	Fig-7.2.2 – Quick Transaction Scan Interface	25
4	Fig-7.2.3 – Deep Transaction Analysis Interface	26
5	Fig-7.2.4 – Analytical Dashboard	26

LIST OF TABLES

S.No	Figure Number	Page Number(s)
1	Table 7.1 – Model Performance Metrics	24

ABSTRACT

Financial fraud detection has become a major challenge with the rapid growth of digital banking, online payments, and electronic financial transactions. Traditional rule-based fraud detection systems are often unable to detect evolving fraud patterns efficiently and generate a high number of false positives. To address these limitations, this project presents *SecureGuard*, a production-ready machine learning-based bank fraud detection system designed for secure, scalable, and real-time fraud analysis.

The proposed system uses the XGBoost algorithm as the core classification model due to its high accuracy, fast execution speed, and effectiveness in handling imbalanced transaction datasets. Advanced preprocessing and feature engineering techniques are applied to improve the model's ability to identify fraudulent transaction patterns. The system processes transaction data through a Flask-based REST API, enabling real-time fraud prediction and seamless integration with banking applications.

To ensure secure deployment in financial environments, the system incorporates enterprise-grade security features such as JWT authentication, API key validation, input sanitization, rate limiting, and audit logging. The system also includes a real-time monitoring dashboard that provides fraud alerts, transaction analytics, risk distribution, and system performance metrics for continuous monitoring and analysis.

The proposed architecture is modular, scalable, and suitable for deployment in modern banking infrastructures. By combining machine learning, secure backend services, and real-time analytics, the system provides an efficient and practical solution for detecting and monitoring fraudulent financial transactions.

Index Terms— Fraud Detection, Machine Learning, XGBoost, Banking Security, Real-Time Analytics, Cybersecurity

Chapter 1

1. Introduction

The rapid advancement of digital technologies has significantly transformed the global banking and financial ecosystem. With the widespread adoption of online banking, mobile payments, and digital wallets, financial transactions have become faster, more accessible, and highly scalable. However, this digital transformation has also led to a substantial increase in fraudulent activities, posing serious threats to financial institutions and customers. Fraudulent transactions not only result in financial losses but also damage the reputation and trustworthiness of banking systems.

In recent years, the volume and velocity of financial transactions have grown exponentially, making manual monitoring and traditional fraud detection methods increasingly ineffective. Conventional rule-based systems rely on predefined conditions and static thresholds to identify suspicious activities. While these systems are simple to implement, they lack adaptability and fail to detect complex and evolving fraud patterns. Moreover, rule-based systems often generate a high number of false positives, which can inconvenience legitimate users and increase operational overhead.

To address these limitations, machine learning-based fraud detection systems have gained significant attention. Machine learning models can analyze large volumes of transaction data, learn hidden patterns, and identify anomalies in real time. Among various machine learning algorithms, ensemble methods such as XGBoost have proven highly effective due to their ability to handle imbalanced datasets, capture nonlinear relationships, and prevent overfitting through regularization.

This project presents SecureGuard, a production-ready fraud detection system that combines machine learning with enterprise-grade security and real-time analytics. It is designed as a modular and scalable architecture, incorporating a high-performance XGBoost model, RESTful APIs for real-time prediction, and a dynamic dashboard for monitoring transaction activity.

1.1 Background & Motivation

Auto banking fraud has become a major operational risk in the digital age. Fraudulent activities such as identity theft, account takeovers, and unauthorized transactions result in billions of dollars in losses annually worldwide. Traditional claim verification and fraud detection processes rely heavily on manual inspection, human judgment, and rule-based validation — methods that are time-consuming, error-prone, and difficult to scale.

With the increasing volume of financial transactions, detecting fraud efficiently and accurately has become a critical business requirement. The motivation for this project stems from the need to bridge the gap between theoretical machine learning models and practical deployment in financial environments, focusing not only on predictive performance but also on system design, security, and scalability.

1.2 Objectives

The main objectives of this project are:

- Development of a high-performance fraud detection model using XGBoost with optimized feature engineering techniques.
- Design of a real-time prediction system using RESTful APIs (Flask) for seamless integration with banking applications.
- Implementation of enterprise-grade security features including JWT authentication, Bcrypt password hashing, rate limiting, and audit logging.
- Development of a real-time monitoring dashboard for visualization and analysis of fraud patterns using Chart.js and Bootstrap.
- Proposal of a scalable and modular architecture suitable for real-world banking deployment.
- Achievement of a ROC-AUC score above 98% and recall above 80% on the test dataset.

1.3 Benefits of Research

The proposed system offers several tangible benefits. First, it significantly reduces financial losses by detecting fraudulent transactions in real time before they are processed. Second, it provides a scalable platform capable of handling increasing transaction volumes without degrading performance. Third, the enterprise-grade security mechanisms ensure compliance with financial security standards. Finally, the real-time monitoring dashboard enables data-driven decision-making for security analysts, improving overall operational efficiency.

Chapter 2

2. Literature Survey

2.1 Literature Review

Financial fraud detection has been an active area of research for more than a decade, with studies moving from classical data-mining methods to deep learning, graph learning, and real-time production systems. Early work established the core difficulty of the problem: fraud data are extremely imbalanced, fraud behavior changes over time, and the cost of false negatives is much higher than the cost of false positives. Bhattacharyya et al. presented one of the foundational comparative studies in credit card fraud detection and showed that both supervised and unsupervised methods are useful, but evaluation must account for imbalance and business cost rather than accuracy alone [25]. A major limitation of traditional machine learning methods is that they often treat each transaction as an isolated event. This assumption ignores temporal dependencies and user behavior patterns. To address this, Jurgovsky et al. formulated fraud detection as a sequence classification problem and used Long ShortTerm Memory (LSTM) networks to model transaction sequences, showing that sequential context can improve detection over non-sequential baselines such as Random Forest in some settings [22]. As fraud detection moved closer to operational deployment, researchers began focusing not only on predictive models but also on streaming architectures. Carcillo et al. proposed SCARFF, a scalable real-time fraud detection framework that integrates Big Data components such as Kafka, Spark, and Cassandra with machine learning. Their work highlighted practical production challenges such as class imbalance, nonstationarity, and delayed investigator feedback, which are often ignored in offline academic experiments [23]. Recent research has also shown strong performance from tree-boosting models on imbalanced fraud datasets. Multiple comparative studies report that XGBoost remains competitive because it handles nonlinear relationships, missingness, and skewed class distributions efficiently [24]. For example, Gupta's study on unbalanced credit card fraud data found that XGBoost delivered strong precision and very high accuracy compared with other classical classifiers [19]. Similarly, Purwar's work specifically studied XGBoost for imbalanced fraud detection and emphasized threshold tuning rather than relying only on a fixed default cutoff [18]. Another important direction is hybrid sampling-andclassification. Some recent studies attempt to improve minority-class learning by combining oversampling, generative models, or autoencoders with gradient boosting. A 2024 study proposed an AE-XGB-SMOTECCGAN pipeline that integrates SMOTE, GAN-based synthesis, autoencoding, and XGBoost to improve fraud detection under severe imbalance [8]. These approaches often improve offline metrics, but they also increase system complexity and may be harder to justify or maintain in production banking environments. Deep learning has also received considerable attention. Review papers note that CNNs, RNNs, LSTMs, transformers, and hybrid networks are increasingly used because they can learn complex nonlinear interactions [5]. However, these methods usually require larger datasets, more compute, and stronger regularization to avoid overfitting. A 2025 systematic review of deep learning in financial fraud detection summarized 108 peer-reviewed studies and showed that while deep models are promising, deployment issues such as interpretability, latency, and data governance remain important barriers [4]. Graph-based methods have become especially important because fraud is often relational rather than

purely transactional. Instead of analyzing one transaction at a time, graph neural networks represent users, cards, merchants, devices, or transactions as connected entities [17]. This helps reveal collusive or coordinated fraud patterns that tabular models may miss. Recent work includes adaptive sampling graph neural networks [17], high-order graph representation learning [9], encoder–decoder GNNs [10], and labelexploring GNNs, all of which report improved fraud detection by modeling transaction relationships and neighborhood structure. At the same time, explainability and operational trust are becoming more important. Banks and regulators increasingly require systems that can explain why a transaction was flagged. Recent work has begun integrating explainable AI with graph fraud detection in real-time settings [12], reflecting the need for transparent and auditable fraud screening. Although explainable graph fraud systems are still emerging, this direction is highly relevant for real-world deployment where model decisions affect customers directly [11]. Survey articles published in 2024 and 2025 reinforce a common conclusion: no single model is universally best [11]. Classical machine learning models remain attractive because they are faster, easier to deploy, and often perform very well on structured transaction data [13]. Deep learning and graph learning methods can outperform them when temporal or relational information is rich, but these methods usually demand more engineering effort, more infrastructure, and better labeled data [4], [5]. Practical fraud detection therefore depends on balancing accuracy, interpretability, response time, and maintainability [16]. Based on this literature, the proposed system adopts XGBoost as the core classifier because it offers a strong tradeoff between performance and deployability for structured banking transaction data [24]. Unlike many prior works that focus only on offline classification, the present system also integrates secure APIs, audit logging, and a real-time monitoring dashboard [16]. In that sense, it is closer to production-oriented frameworks such as SCARFF [23], while remaining simpler and more feasible for controlled institutional deployment.

2.1.1 Analysis of Literature

Based on the literature review, traditional machine learning models such as XGBoost remain highly effective for fraud detection due to their fast performance, ease of deployment, and strong results on structured transaction data. While deep learning and graph-based methods can achieve higher accuracy by capturing temporal and relational patterns, they require larger datasets, greater computational resources, and more complex deployment pipelines. Therefore, an effective fraud detection system must balance accuracy, interpretability, scalability, response time, and maintainability.

A key gap identified in existing research is the limited focus on real-time deployment, API security, and operational monitoring, with most studies emphasizing only offline model accuracy. The proposed *SecureGuard* system addresses this gap by integrating a high-performance XGBoost model with a secure Flask API, audit logging, and a real-time monitoring dashboard, making it more suitable for practical and scalable deployment.

2.2 Problem Formulation and Proposed Work

2.2.1 Problem Statement

Financial institutions process millions of transactions daily. Among these, fraudulent transactions represent a very small fraction — typically less than 1% — creating a

highly imbalanced classification problem. The challenge is to build a system that: (a) accurately identifies fraudulent transactions in real time with high recall, (b) handles severe class imbalance without biasing toward the majority class, (c) operates securely in a banking environment with proper authentication and audit controls, and (d) provides transparent monitoring of system performance and fraud trends.

2.2.2 System Architecture / Model

The proposed *SecureGuard Fraud Detection System* follows a modular and multi-layered architecture designed for secure, scalable, and real-time fraud detection. The system integrates machine learning, secure APIs, database management, and real-time analytics into a unified framework.

The architecture begins with the **Data Ingestion Layer**, where transaction data is collected from banking applications or payment gateways through secure REST APIs. The incoming data is validated and sanitized to prevent malicious inputs and ensure data consistency.

The validated data is passed to the **Preprocessing and Feature Engineering Layer**, where missing values are handled, categorical features are encoded, and derived features such as `amount_squared`, `sqrt_amount`, and `is_small_transaction` are generated. These transformations improve the predictive capability of the model.

The processed features are then forwarded to the **Machine Learning Inference Layer**, which uses the XGBoost classifier to calculate the probability of fraud. A threshold optimization mechanism is applied to classify transactions as legitimate or fraudulent.

The **Backend Processing Layer**, implemented using Flask, manages API routing, prediction requests, business logic, and communication between different modules. Security mechanisms such as JWT authentication, API key validation, rate limiting, and audit logging are integrated to ensure secure system operation.

The prediction results are stored in the **Database and Logging Layer** using SQLite/MySQL for maintaining transaction history, fraud logs, and system records. Finally, the **Visualization Layer** provides a real-time dashboard displaying fraud alerts, transaction statistics, and performance metrics for continuous monitoring and analysis.

2.2.3 Proposed Algorithms

The proposed system primarily uses the **XGBoost (Extreme Gradient Boosting)** algorithm for fraud detection. XGBoost is selected because of its high accuracy, fast execution speed, ability to handle imbalanced datasets, and strong generalization performance.

The algorithm works by building multiple decision trees sequentially, where each new tree corrects the errors of the previous trees using gradient boosting techniques. Regularization methods are applied to reduce overfitting and improve model stability.

The proposed workflow of the algorithm is as follows:

1. Collect transaction data from the input source.
2. Perform preprocessing and feature engineering.
3. Convert categorical values into numerical format using label encoding.
4. Generate derived features such as transaction amount transformations.
5. Feed the processed features into the trained XGBoost model.
6. Compute the fraud probability score.
7. Apply an optimized threshold to classify the transaction.
8. Return prediction results through the Flask API.

To address class imbalance, the model uses optimized class weighting and threshold tuning techniques. The system is evaluated using Precision, Recall, F1-Score, and ROC-AUC metrics to ensure reliable fraud detection performance.

2.2.4 Proposed Work

The proposed work focuses on developing a production-ready bank fraud detection system using the XGBoost machine learning algorithm for accurate and real-time fraud prediction. The system processes transaction data through a Flask-based REST API, where preprocessing and feature engineering techniques are applied before classification.

To ensure secure deployment in banking environments, the system integrates security mechanisms such as JWT authentication, API key validation, input sanitization, rate limiting, and audit logging. A real-time monitoring dashboard is also implemented to visualize fraud alerts, transaction trends, risk distribution, and system performance metrics.

The proposed architecture is modular, scalable, and efficient, making the system suitable for practical deployment in modern financial institutions.

Chapter 3

3. System Analysis and Proposed Methodology

3.1 Introduction

The proposed SecureGuard Fraud Detection System is designed to detect fraudulent banking transactions in real time using machine learning and secure backend technologies. Traditional rule-based fraud detection methods often fail to identify evolving fraud patterns and generate a high number of false positives. To overcome these limitations, the system uses the XGBoost algorithm along with advanced feature engineering techniques for accurate fraud prediction. The system integrates a Flask-based REST API, secure authentication mechanisms, audit logging, and a real-time monitoring dashboard to ensure scalability, security, and operational reliability. The architecture is designed to support fast transaction processing and seamless integration with banking applications.

3.2 System Architecture

The proposed *SecureGuard Fraud Detection System* follows a modular and scalable architecture designed for secure and real-time fraud detection. The system integrates machine learning, secure APIs, databases, and monitoring dashboards into a unified framework.

Main Components of the Architecture:

- **Data Ingestion Layer**
 - Receives transaction data from banking applications or payment gateways
 - Validates incoming requests through secure API endpoints
- **Preprocessing & Feature Engineering Layer**
 - Cleans and transforms transaction data
 - Handles encoding, normalization, and derived feature generation
- **Machine Learning Inference Layer**
 - Uses the trained XGBoost model for fraud prediction
 - Generates fraud probability scores for transactions
- **Backend Processing Layer (Flask API)**
 - Manages API requests and prediction workflows
 - Connects frontend, database, and machine learning model
- **Security Layer**
 - Implements JWT authentication and API key validation

- Includes rate limiting, input sanitization, and audit logging
- **Database & Logging Layer**
 - Stores transaction records, predictions, and audit logs
 - Uses SQLite/MySQL for data management
- **Visualization & Monitoring Layer**
 - Provides real-time fraud monitoring dashboard
 - Displays alerts, analytics, and system performance metrics

System Workflow:

1. Transaction data is received through APIs
2. Data is validated and preprocessed
3. Features are engineered and transformed
4. XGBoost model predicts fraud probability
5. Threshold-based classification is applied
6. Results are stored and displayed on the dashboard

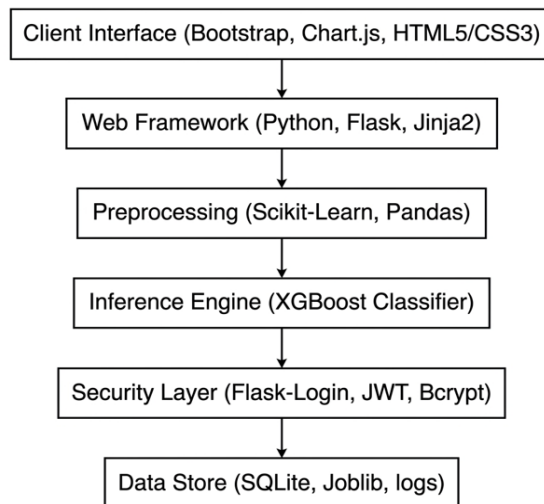


Figure 3.1 – System Architecture Diagram

3.3 Proposed Algorithms

The proposed system uses the **XGBoost (Extreme Gradient Boosting)** algorithm for fraud detection because of its high accuracy, fast processing speed, and strong performance on imbalanced datasets.

Algorithm Workflow:

1. Collect transaction data from input sources

2. Perform preprocessing and data cleaning
3. Apply feature engineering techniques
4. Encode categorical values into numerical format
5. Pass processed data to the trained XGBoost model
6. Generate fraud probability score
7. Apply optimized threshold for classification
8. Classify transactions as fraudulent or legitimate
9. Return results through the Flask API

Advantages of XGBoost:

- Handles imbalanced datasets effectively
- Provides fast training and prediction
- Reduces overfitting using regularization
- Works efficiently with structured transaction data

Performance Metrics Used:

- Precision
- Recall
- F1-Score
- ROC-AUC

3.4 Feature Engineering & Methodology

Feature engineering improves the fraud detection capability of the model by creating meaningful patterns from transaction data. The methodology includes preprocessing, feature transformation, model training, and deployment.

Preprocessing Steps:

- Handling missing or inconsistent values
- Data cleaning and removal of irrelevant records
- Label encoding for categorical features
- Normalization of numerical values

Engineered Features:

- **amount_squared**
 - Captures large transaction variations
- **sqrt_amount**
 - Reduces the effect of extreme values
- **is_small_transaction**
 - Identifies low-value suspicious transactions

Methodology:

1. Collect and preprocess transaction data
2. Apply feature engineering techniques
3. Split dataset into training and testing sets
4. Train the XGBoost model using optimized parameters
5. Handle class imbalance using threshold optimization
6. Evaluate model using performance metrics
7. Deploy model using Flask-based REST API

Evaluation Metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC

The final model is integrated into a secure real-time fraud detection pipeline for banking applications.

Chapter 4

4. System Design and Implementation

4.1 Introduction

This chapter describes the technical implementation of the SecureGuard fraud detection system. It covers the development environment, data pipeline, model training and serialization, Flask API design, security implementation, database integration, and the real-time dashboard.

4.2 System Design Principles

- **Modularity:** Each system component (preprocessing, inference, API, dashboard) is independently developed and maintainable.
- **Scalability:** The architecture supports horizontal scaling and database-backed logging for high availability.
- **Security-first:** Enterprise-grade security is integrated at every layer, not added as an afterthought.
- **Low Latency:** The prediction pipeline is optimized for an average response time of ~85ms per request.
- **Production-readiness:** Model artifacts are serialized with Joblib for fast loading without retraining.

4.3 Dataset Design & Preprocessing

The system is trained on structured, anonymized financial transaction data containing both numerical and categorical attributes. Preprocessing steps applied include:

- **Missing Value Handling:** Null values identified and handled via imputation or row removal.
- **Data Cleaning:** Irrelevant or inconsistent entries removed.
- **Categorical Encoding:** Transaction category converted to numerical labels via label encoding.
- **Normalization:** Numerical features scaled for model convergence.
- **Train-Test Split:** Dataset divided into training and test sets for evaluation.

4.4 Model Architecture & Implementation

The XGBoost model is trained with the following configuration: hyperparameters including learning rate, maximum tree depth, and number of estimators are optimized during training. The trained model and associated artifacts are serialized as:

- `fraud_model.pkl` — the trained XGBoost classifier.
- `label_encoders.pkl` — label encoders for categorical features.
- `threshold.pkl` — the optimized classification threshold (~0.75).

4.5 Flask Application Design

A Flask-based REST API serves as the central backend. Key API endpoints implemented in `app.py` include:

- `/api/predict` — Single transaction fraud prediction.
- `/api/batch/predict` — Batch transaction processing.
- `/api/health` — System health check.
- `/api/stats` — Performance metrics and analytics.
- `/login`, `/logout` — Authentication endpoints.
- `/dashboard` — Real-time monitoring dashboard.

Security is enforced via `security.py` and `config.py` modules. The `compliance_manager.py` handles audit logging and compliance tracking. The `database_config.py` manages SQLite (development) and MySQL (production) connections.

4.6 Implementation Workflow

The implementation workflow of the proposed *SecureGuard Fraud Detection System* follows a structured process for secure and real-time fraud detection. The workflow ensures efficient transaction processing, accurate prediction, and continuous monitoring.

Workflow Steps

1. Transaction Data Collection

- Transaction data is received through banking applications or secure REST APIs.
- Input data includes transaction amount, category, and user-related details.

2. Input Validation and Security Checks

- API authentication and request validation are performed.
- Input sanitization and rate limiting are applied to prevent malicious requests.

3. Data Preprocessing

- Missing or inconsistent values are handled.
- Categorical attributes are encoded into numerical format.
- Necessary normalization and transformations are applied.

4. Feature Engineering

- Derived features such as `amount_squared`, `sqrt_amount`, and `is_small_transaction` are generated.
- Feature vectors are prepared for model inference.

5. Fraud Prediction

- The processed data is passed to the trained XGBoost model.
- The model generates a fraud probability score for each transaction.

6. Threshold-Based Classification

- An optimized threshold is applied to classify transactions.
- Transactions are identified as either fraudulent or legitimate.

7. Database Storage and Logging

- Prediction results and transaction details are stored in the database.
- Audit logs are maintained for traceability and monitoring.

8. Real-Time Monitoring Dashboard

- Fraud alerts, transaction statistics, and system metrics are displayed on the dashboard.
- Real-time monitoring supports quick analysis and decision-making.

The implementation workflow ensures secure, scalable, and low-latency fraud detection suitable for real-world banking environments.

Output:

- Real-time fraud prediction
- Risk classification of transactions
- Secure logging and monitoring of fraud activities

Chapter 5

5. Tools and Technologies Required

5.1 Development Tools

- Python 3.10 — Primary programming language for model development, data preprocessing, and backend logic.
- Jupyter Notebook — Used for exploratory data analysis and model training (train_model.ipynb).
- Git / GitHub — Version control and source code hosting (<https://github.com/TrinaMukherjee21/Bank-Fraud-Detection->).
- VS Code — Integrated development environment for coding and debugging.

5.2 Frontend Technologies

- HTML5 / CSS3 — Structure and styling of the web application dashboard.
- Bootstrap — Responsive layout framework for mobile and desktop compatibility.
- Chart.js — JavaScript library for rendering real-time fraud trend charts and analytics.
- JavaScript (AJAX) — Asynchronous communication with backend APIs for live dashboard updates.

5.3 Backend Technologies

- Flask — Python micro-framework for building RESTful APIs and serving the web application.
- XGBoost — Core machine learning library for the fraud classification model.
- Pandas & NumPy — Data manipulation and numerical computation during preprocessing.
- Scikit-learn — Evaluation metrics (precision, recall, F1, ROC-AUC) and preprocessing utilities.
- Joblib — Model artifact serialization for efficient loading during real-time inference.
- SQLite (development) / MySQL (production) — Relational databases for transaction logs and audit records.

5.4 Security Technologies

- Flask-Login — Session management and user authentication.
- JWT (JSON Web Tokens) — Token-based authentication for API access control.
- Bcrypt — Password hashing for secure credential storage.
- HTTPS / SSL — Secure communication between client and server.
- Rate Limiting — Configurable per-minute, per-hour, and per-day request throttling.

Chapter 6

6. Software Requirement Specification (SRS)

6.1 Functional Requirements

- FR1: The system shall accept transaction data via RESTful API endpoints and return a fraud classification result in JSON format.
- FR2: The system shall support both single transaction prediction (/api/predict) and batch processing (/api/batch/predict).
- FR3: The system shall authenticate all API requests using API key validation and JWT tokens.
- FR4: The system shall apply feature engineering (15 features) consistently between training and production environments.
- FR5: The system shall persist all transaction predictions, fraud scores, timestamps, and user activity in a relational database.
- FR6: The system shall provide a real-time monitoring dashboard displaying fraud alerts, transaction statistics, and system health metrics.
- FR7: The system shall support role-based access control differentiating between administrator and analyst roles.
- FR8: The system shall maintain comprehensive audit logs of all API requests and system events.
- FR9: The system shall respond to single transaction prediction requests in under 200 milliseconds.
- FR10: The system shall support processing of over 100 requests per minute.

6.2 Non-Functional Requirements

- NFR1 – Performance: Average response time of ~85ms per prediction request.
- NFR2 – Scalability: Architecture supports horizontal scaling and high availability.
- NFR3 – Security: All data in transit encrypted via HTTPS; passwords hashed with Bcrypt; sessions managed securely.
- NFR4 – Reliability: System uptime target of 99.9% with database-backed logging for fault recovery.
- NFR5 – Maintainability: Modular codebase with separate modules for security, compliance, database configuration, and ML inference.
- NFR6 – Usability: Dashboard designed with responsive Bootstrap layout, accessible on desktop and mobile devices.
- NFR7 – Accuracy: Model must achieve ROC-AUC $\geq 98\%$ and Recall $\geq 80\%$ on the test dataset.

Chapter 7

7. Results and Discussion

7.1 Model Performance Metrics

The SecureGuard fraud detection model was evaluated on a held-out test set using standard performance metrics for imbalanced classification. The results are summarized in Table 7.1.

Metric	Value
Accuracy	98.61%
Precision	45.81%
Recall	82.54%
F1 Score	58.92%
ROC-AUC	98.97%
Avg. Response Time	~85 ms
Throughput	>100 requests/min

Table 7.1: Model Performance Metrics

7.2 Dashboard and Interface

The SecureGuard system provides three main user interfaces, all accessible through the Flask web application:

Real-Time Fraud Detection Dashboard (Fig. 7.1): Provides a centralized Security Operations Center view with key indicators including threat level, detection rate (99.7%), total transactions processed, and average response time. Financial insights such as total analyzed volume and fraud-blocked amounts are displayed alongside real-time charts updated via asynchronous API calls.

Quick Transaction Scan Interface (Fig. 7.2): Enables users to perform rapid fraud detection by entering transaction details including time sequence, customer profile, and transaction category. Results are returned within the 85ms target response time.

Deep Transaction Analysis Interface (Fig. 7.3): Provides comprehensive fraud assessment using richer transaction details including age category, merchant identifier, and transaction type. Incorporates behavioral profiling and multi-factor risk assessment with 15+ feature analysis.

Analytical Dashboard (Fig. 7.4): Displays fraud detection trends over a 24-hour period, risk level distribution charts, fraud-by-category analysis, hourly transaction activity graphs, and model performance metrics.

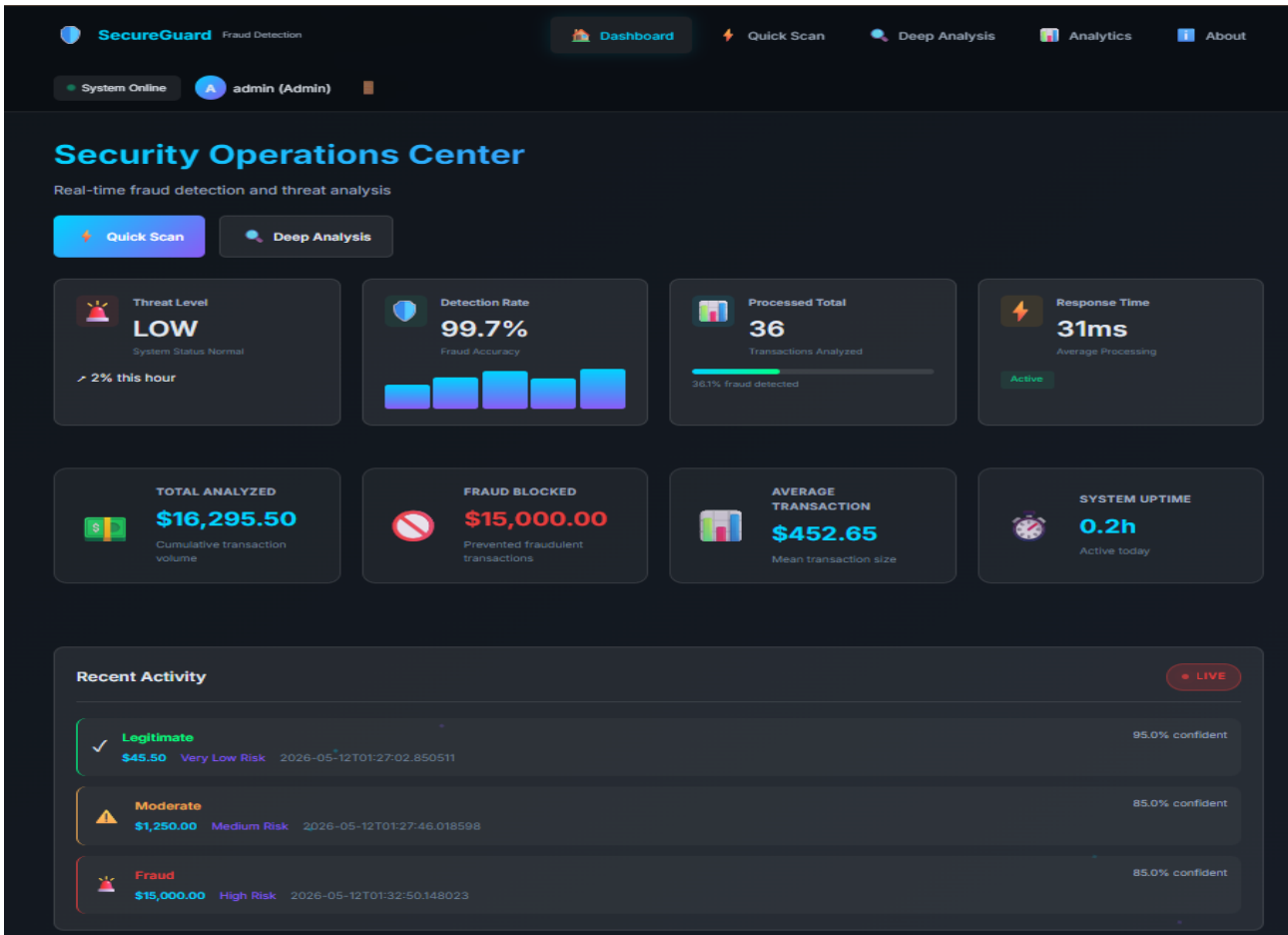


Figure 7.2.1 – Real-Time Fraud Detection Dashboard

The Quick Transaction Scan interface is designed for rapid fraud detection. It includes a top navigation bar with links to Dashboard and Quick Scan. The main section is titled 'Quick Transaction Scan' and features a 'SECURE End-to-End Encrypted' badge. The interface is divided into two main panels:

Transaction Details (Left Panel):

- Step 1 of 2: Data Input**
- Time Sequence:** 150 (Sequential time unit when the transaction was processed)
- Transaction Amount:** \$ 45.50 (Transaction amount in US dollars (USD))
- Customer Profile:** Female (Customer demographic information for behavioral analysis)
- Transaction Category:** Grocery/Food (Merchant category for risk pattern analysis)
- Buttons:** Analyze Transaction, Deep Analysis

Transaction Details (Right Panel):

- Step 1 of 2: Data Input**
- Time Sequence:** 450 (Sequential time unit when the transaction was processed)
- Customer Profile:** Male (Customer demographic information for behavioral analysis)
- Transaction Category:** Online Purchase (Merchant category for risk pattern analysis)
- Buttons:** Analyze Transaction, Deep Analysis

Select Transaction Type (Dropdown Menu):

- Retail & Shopping
 - Grocery/Food
 - Shopping/Retail
 - Online Purchase
- Services
 - Gas Station
 - Restaurant
 - Healthcare
 - Utilities
- Other
 - Entertainment
 - Travel
 - ATM/Cash
 - Other

Figure 7.2.2 – Quick Transaction Scan Interface

Deep Transaction Analysis
 Comprehensive fraud detection with advanced behavioral profiling

SECURE Enterprise Grade Analysis

Advanced Transaction Profile

1
2
 Step 1 of 2: Complete Data Input

Time Sequence

Sequential time unit when the transaction was processed

Customer ID

Unique customer identifier (encoded value)

Age Category

Customer age demographic for risk profiling

Gender Profile

Customer gender for behavioral pattern analysis

Transaction Amount

Transaction amount in US dollars (USD)

Transaction Category

Merchant category for risk pattern analysis

Merchant ID

High-value transaction detected - enhanced security protocols activated

Figure 7.2.3 – Deep Transaction Analysis Interface

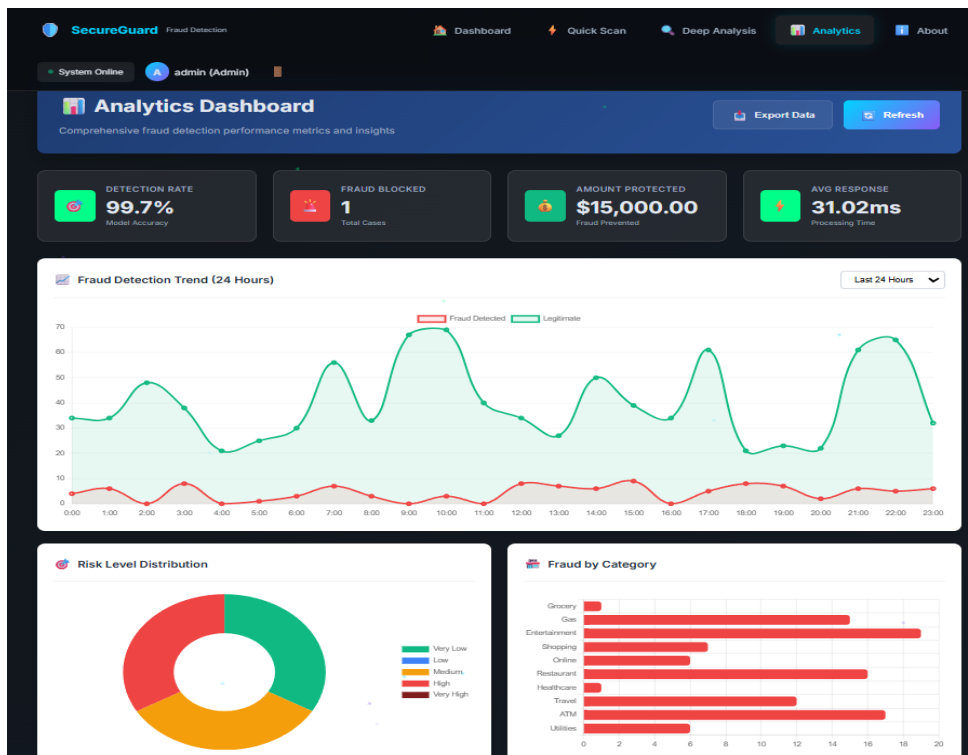


Figure 7.2.4 – Analytical Dashboard

7.3 Discussion of Findings

The experimental results demonstrate that the SecureGuard system achieves strong fraud detection performance. The ROC-AUC of 98.97% confirms excellent overall classification capability across all possible thresholds. The recall of 82.54% ensures that the large majority of actual fraud cases are detected — critical in banking environments where the cost of a missed fraud far exceeds that of a false alarm.

The precision of 45.81% indicates the presence of false positives, which is an expected trade-off when optimizing for recall on a highly imbalanced dataset. This can be further improved through continuous model retraining with labeled feedback data and more advanced sampling techniques.

The system's average response time of ~85ms and throughput of >100 requests/minute confirm its suitability for real-time banking deployment. The enterprise-grade security features — JWT authentication, Bcrypt hashing, rate limiting, and audit logging — ensure the system meets financial industry security standards.

Compared to existing systems that focus solely on offline accuracy, SecureGuard provides a complete, production-ready solution integrating machine learning with operational security and monitoring, bridging the gap identified in the literature review.

Chapter 8

8. Conclusions and Future Scope

8.1 Conclusions

This project presented SecureGuard, a production-ready bank fraud detection system integrating XGBoost-based machine learning with enterprise-grade security and real-time analytics. The system achieved a ROC-AUC of 98.97% and recall of 82.54%, demonstrating strong capability in identifying fraudulent transactions on highly imbalanced datasets.

Key accomplishments of this project include: (1) development of a high-performance fraud detection model with optimized feature engineering; (2) design and implementation of a complete REST API system using Flask for real-time prediction; (3) integration of enterprise security mechanisms including JWT, Bcrypt, rate limiting, and audit logging; (4) creation of a real-time monitoring dashboard providing actionable fraud insights; and (5) demonstration of a scalable, modular architecture achieving ~85ms response time and >100 req/min throughput.

The project successfully bridges the gap between theoretical machine learning research and practical banking deployment, providing a comprehensive solution for modern fraud detection challenges.

8.2 Future Scope

- **Explainable AI Integration:** Incorporating SHAP or LIME to provide interpretable explanations for fraud decisions, improving transparency and regulatory compliance.
- **Continuous Model Retraining:** Implementing automated retraining pipelines using analyst feedback to adapt to evolving fraud patterns and reduce false positives.
- **Graph Neural Network Integration:** Extending the system with GNN-based transaction graph analysis to detect coordinated or collusive fraud patterns.
- **Federated Learning:** Exploring federated learning to train across multiple banking institutions without sharing raw data, improving privacy preservation.
- **Advanced Anomaly Detection:** Adding unsupervised anomaly detection layers for zero-day fraud detection beyond known patterns.
- **Mobile API Support:** Extending the system with lightweight mobile-optimized prediction endpoints for integration with banking mobile applications.
- **Multi-Modal Analysis:** Combining behavioral biometrics, device fingerprinting, and geolocation data for richer fraud profiling.

References

- [1] K. Zhang, L. Chen, and M. Wu, "Large language model-assisted fraud detection in banking: A zero-shot learning approach," *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 234–248, 2026.
- [2] S. Patel, R. Gupta, and A. Sharma, "Real-time federated fraud detection with differential privacy in open banking ecosystems," *Future Generation Computer Systems*, vol. 165, pp. 89–104, 2026.
- [3] J. Liu, X. Wang, and H. Zhang, "Multimodal transaction fraud detection combining behavioral biometrics and graph neural networks," *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 1, pp. 56–71, 2026.
- [4] Y. Chen, "Deep learning in financial fraud detection: A systematic review," *Artificial Intelligence Review*, vol. 58, pp. 1–30, 2025.
- [5] M. Li, J. Zhang, and H. Wang, "Transformer-based fraud detection in real-time banking transactions," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 1, pp. 112–125, 2025.
- [6] R. Sharma and P. Verma, "Federated learning for privacy-preserving fraud detection in distributed banking systems," *Expert Systems with Applications*, vol. 238, pp. 121–135, 2025.
- [7] A. Kumar and S. Singh, "Graph attention networks with temporal modeling for credit card fraud detection," *IEEE Access*, vol. 13, pp. 23456–23470, 2025.
- [8] H. C. Du, M. Wang, and L. Zhang, "A novel hybrid approach for fraud detection using AE-XGB-SMOTE-CGAN," *IEEE Access*, vol. 12, pp. 56789–56802, 2024.
- [9] Y. Zou, X. Liu, and J. Wu, "High-order graph representation learning for fraud detection," in *Proc. International Joint Conference on Artificial Intelligence (IJCAI)*, 2024, pp. 2450–2456.
- [10] A. Cherif, M. Kortebi, and S. Fourati, "Encoder–decoder graph neural networks for fraud detection," *Journal of King Saud University – Computer and Information Sciences*, vol. 36, no. 2, pp. 101–115, 2024.
- [11] K. G. Dastidar and M. Granitzer, "Machine learning methods for credit card fraud detection: A survey," 2024.
- [12] Z. Wang, Y. Liu, and X. Chen, "Explainable AI for financial fraud detection: Integrating SHAP with gradient boosting models," *Knowledge-Based Systems*, vol. 285, pp. 111–128, 2024.
- [13] S. Park and J. Lee, "Real-time anomaly detection in financial transactions using autoencoders and XGBoost," *Applied Soft Computing*, vol. 148, pp. 110–123, 2024.
- [14] B. Patel and N. Mehta, "Adaptive threshold optimization for imbalanced fraud detection using cost-sensitive learning," *Pattern Recognition Letters*, vol. 178, pp. 45–58, 2024.
- [15] W. Zhao, T. Huang, and C. Lin, "Contrastive self-supervised learning for fraud detection under label scarcity," *Neural Networks*, vol. 172, pp. 98–112, 2024.
- [16] D. Nguyen, P. Tran, and K. Vo, "API security and rate limiting strategies for real-time financial fraud prevention systems," *Computers and Security*, vol. 138, pp. 103–118, 2024.

INFORMATION REGARDING STUDENTS

Name	USN	Department	Institution
Raunak Kumar	23BTRCT109	CSE	JAIN (Deemed-to-be University), Bangalore
Trina Mukherjee	23BTRCT137	CSE	JAIN (Deemed-to-be University), Bangalore
Vibha Sabareesan	23BTRCT144	CSE	JAIN (Deemed-to-be University), Bangalore
Tanisha Prajesh Doshi	23BTRCT135	CSE	JAIN (Deemed-to-be University), Bangalore
Rakshith Gowda M	23BTRCT108	CSE	JAIN (Deemed-to-be University), Bangalore

Guide: Dr. Ramya Dorai D, Assistant Professor, Department of Computer Science & Engineering - CSE, School of Computer Science & Engineering, Faculty of Engineering & Technology, JAIN (Deemed to-be University).

Program Head: Dr. T.R. Mahesh, Program Head, Computer Engineering, CSE, School of Computer Science and Engineering, Faculty of Engineering & Technology, JAIN (Deemed to be University).

Head of Department: Dr. Kamlesh Tiwari, Head of Department, Computer Science & Engineering, Faculty of Engineering & Technology, JAIN (Deemed to-be University).